Sciex ZT-Scan DIA: Disk-Spilled Meta-Scan Collapse

Lowering peak RSS while staying byte-identical — design + code examples

Pioneer.jl branch `feat/zt-batched-collapse`

2026-07-21

Abstract

The in-memory "batched" collapse (previous plan) was a dead end: contiguous scan batches change the deconvolution's per-precursor warm-start order, so results were *not* byte-identical (−0.1% prec / −0.9% PG), and holding all per-batch raw tables plus borrow-copies actually *raised* peak (+4.4 GB) and runtime (+84%). This plan takes the opposite approach: **leave the round-robin deconvolution and its thread/scan assignment exactly as on develop** (cache-optimal, and byte-identical raw rows), and instead **spill the raw PSMs to disk in contiguous scan-range bins**, then read and collapse one bin at a time. Peak in-memory PSM footprint drops from the full ∼6.3 GB raw table to ∼one bin plus the accumulating ∼1.1 GB meta table. Byte-identical, because the raw rows are unchanged and the per-bin collapse-with-borrow is already proven bit-identical to the global collapse (the `PIONEER_ZT_DIAG_COMPARE` check: same raw ⇒ same 2,950,279).

1 Why the previous (in-memory contiguous-batch) approach failed

- **Not byte-identical.** `post_design_matrix!` warm-starts each precursor's Huber solve from its converged weight at the previous scan (`initialize_weights!` ← per-thread `precursor_weights`). Re-partitioning scans rewires every warm-start chain ⇒ weights shift near the 10^{-6} emission threshold (33,185,795 → 33,184,306 raw rows). The collapse logic itself was proven correct.
- **Higher peak + slower.** Barrier holds all per-batch raws *and* `batch_tbl = vcat(head,raw,tail)` copies each ⇒ ∼ 2× raw memory; contiguous chunks also load-balance worse than round-robin.

Takeaway: do not touch the deconv partition. Keep round-robin *exactly*.

2 Peak-RSS profiling (what actually drives the peak)

`PIONEER_DIAG_RSS=1` logs current RSS (via `ps`) + high-water at each Main-Search sub-step. Single file, ZT, global path:

Sub-step	current RSS	
pre `library_search` (base + library loaded)	9.95 GB	
post `library_search` (deconv)	31.13 GB	+21.2
post `scan_comp`+`ms1`+permute	32.72 GB	+1.6
post `prepare_psm_features`	**33.94 GB**	+1.2 (**peak**)
post collapse + chromatogram	30.13 GB	−3.8 (collapse frees)
post `train_lgbm`	32.08 GB	

(These are `ps`/`maxrss` numbers, ∼34 GB; the macOS `phys_footprint` peak is 22.4 GB — it excludes file-backed clean pages. The raw PSM table is Julia heap = dirty = counted in both.)

Findings. (1) The peak is at *post-prepare*, where the 33M-row raw table is widest (deconv + `scan_comp` + `ms1` + prepare cols ≈ 9 GB), and the collapse frees −3.8 GB — so removing the raw table from this window lowers peak. (2) The deconv jump is +21 GB, of which the raw table is only ∼6.3 GB; the rest is per-thread deconv scratch (∼8 GB, inherent) *plus the `vcat(fetched...)` transient that copies the raw a second time (∼6.3 GB).* The permute is *not* a driver: removing its 7.45 GB churn left peak unchanged (22.30 → 22.15 GB byte-identical) — it is freed before the peak instant.

Consequence for this plan. The biggest lever is **eliminating the `vcat`**: scatter per-thread tables straight to disk (never build the concatenated 33M table, never run prepare on it). That removes both the vcat copy and the wide raw table from the post-deconv peak. Realistic target: phys-footprint peak 22.4 →∼ 16–18 GB — bounded below by the ∼8 GB deconv scratch.

3 Design: keep round-robin deconv, spill raw to disk, collapse per bin

Five stages; only stages 3–4 are new. Everything is byte-identical.

1. **Deconvolution — UNCHANGED.** Round-robin threaded deconv (develop's `partition_scans` + `process_scans_fused!`). Per-thread output is *scan-disjoint* and contiguous-by-scan (existing invariant).
2. **Per-thread `scan_competition` + `ms1_lookup` — byte-identical.** Because each scan's rows live entirely in one thread, per-scan `scan_competition` per thread equals the current global pass; `ms1_lookup` is per-row. Run them on each thread table before spilling, so spilled rows already carry those feature columns.
3. **Spill to B contiguous scan-range bins.** Scatter each thread table's rows by `scan_idx` into B on-disk Arrow bins, *freeing each thread table as soon as it is scattered* (fetch→scatter→free, one thread at a time) so the full raw never coexists in memory.
4. **Collapse one bin at a time.** For bin b (a contiguous scan range), read bin b plus the $\pm k$ halo slices from the adjacent bins' files, run the proven `collapse_to_metascans(...; own_scans = bin_b_scans)`, append the small meta result, and free the bin. Peak PSM memory = one bin (∼raw/B) + halo + accumulating meta.
5. **Assemble — UNCHANGED downstream.** `vcat` the per-bin metas, `sort` by `(precursor_idx, scan_idx)` to restore global order, then the existing `prepare_psm_features!`, `add_chromatogram_features!`, and LightGBM run on the ∼2.95M-row meta table exactly as today.

4 Byte-identity argument

- Raw rows identical: deconv partition and warm-start chains are untouched.
- `scan_competition` per-thread = global: scans are thread-disjoint, and the feature is per-scan-local (no file-global normalization). `ms1_lookup`/`prepare` are per-row.
- Collapse per-bin-with-borrow = global collapse: proven by `PIONEER_ZT_DIAG_COMPARE` (identical raw ⇒ identical centers). Bins tile the scans; each center is emitted once (`own_scans`); $\pm k$ halo covers every neighbor.
- Final `sort` by `(pid,scn)` reproduces the exact LightGBM row order.

5 Expected effect

	in-memory batched (rejected)	disk-spill (this plan)
Byte-identical	no (−0.9% PG)	**yes**
Peak PSM memory	∼ 2× raw (worse)	∼raw/B + 1.1 GB meta
Deconv partition	contiguous (cache-hostile)	**round-robin (unchanged)**
Extra cost	—	write+read ∼6.3 GB to local disk (IO)

Net: trade ∼6.3 GB of disk write+read for removing the full raw table from the peak resident set. Speed impact is the IO (local SSD; overlap with compute where possible).

6 Code sketch

6.1 Scatter a thread table into on-disk scan-range bins (free as we go)

```
# B contiguous scan-range bin edges over the file's MS2 scans (computed once).
# bin_of[scan_idx] -> 1..B. Each bin is an append-only Arrow writer on disk.
function spill_thread_table!(bin_writers, bin_of, tbl::DataFrame)
    scn = tbl[!, :scan_idx]::Vector{UInt32}
    # group row indices by bin, then append each contiguous slice once
    order = sortperm(scn)                       # tbl is already ~scan-ordered; cheap
    b_prev = 0; lo = 1
    for i in 1:length(order)
        b = bin_of[scn[order[i]]]
        if b != b_prev && b_prev != 0
            append_arrow!(bin_writers[b_prev], @view tbl[order[lo:i-1], :])
            lo = i
        end
        b_prev = b
    end
    append_arrow!(bin_writers[b_prev], @view tbl[order[lo:end], :])
end

# In library_search, replace `result = vcat(fetched...)` for the ZT path with:
for t in tasks                                  # fetch -> scatter -> FREE, one at a time
    tbl = _zt_fetch(t)
    add_scan_competition_features!(tbl)         # per-thread (scans are thread-disjoint)
    add_ms1_lookup_features!(tbl, spectra, search_context, ms_file_idx)
    spill_thread_table!(bin_writers, bin_of, tbl)
    tbl = nothing                               # raw slice freed before the next thread
end
close_all!(bin_writers)                         # B Arrow files on disk; full raw never resident
```

6.2 Collapse bins one at a time, borrowing halo from neighbor files

```
metas = DataFrame[]
for b in 1:B
    bin   = DataFrame(Arrow.Table(bin_path(b)))                    # ~raw/B rows
```

```
    head  = b > 1 ? read_scan_ge(bin_path(b-1), first(bin_scans[b]) - k) :
        nothing
    tail  = b < B ? read_scan_le(bin_path(b+1), last(bin_scans[b])  + k) :
        nothing
    batch = _zt_concat_borrow(head, bin, tail)
    own   = Set{UInt32}(UInt32.(bin_scans[b]))
    push!(metas, collapse_to_metascans(batch, spectra, lib_precs, k;
              bitvec_rank_table = bitvec_rank_table, own_scans = own))
    bin = batch = nothing                                    # bin freed
        each iter
end
result = vcat(metas...)
sort!(result, [:precursor_idx, :scan_idx])                   # global LGBM
    order
# then prepare_psm_features! + add_chromatogram_features! + LightGBM (unchanged)
```

7 Open design choices (for review)

- B **(bin count):** larger $B \Rightarrow$ lower peak, more files/IO. Pick from the profiled peak target (e.g. B=8–16 for ∼0.4–0.8 GB/bin).
- **Scatter concurrency:** per-thread-per-bin files (no locks, $B\times$nthreads files, merged on read) vs shared bin writers with a lock. Former is simpler + faster.
- **Where scatter runs:** inside `library_search` (as sketched) vs a new stage. Keeping it in `library_search` means `process_file!` just receives the meta table.
- **IO overlap:** collapse bin b while spilling continues, if we spill in scan order (a bin is complete once all threads pass its range). Optional; adds complexity.
- **Temp location + cleanup:** reuse the existing `temp_data` dir; delete bins after collapse (respect `delete_temp`).
- **Dispatch:** gate on a config field (`acquisition.zt_metascan_k`), not the env var, so non-ZT runs never touch any of this (deferred until the approach is chosen).

8 Verification plan

1. Byte-identity: single-file collapse stays `33,185,795` → `2,950,447`; meta dump content-identical to `meta_global`; final IDs `26,167 / 4,213`.
2. Peak: `PIONEER_DIAG_RSS` markers show the per-bin collapse peak < the current raw-table peak; macOS `time -l` footprint drops.
3. Speed: warm-process timing (`warm_driver.jl`) — quantify the IO cost.
4. Non-ZT unchanged: gated path; verify an Astral/Exploris run is bit-identical.